

MCP CONTEXT LAYER
DEMO

Unified Agent Memory Across VS Code, OpenCode & OpenWork

Demonstrating seamless context handoffs and zero-loss state persistence across isolated AI agent environments using SurrealDB and Model Context Protocol.

 VS Code Copilot →  OpenCode →  OpenWork

The AI Context Silo Problem



Isolated Memory

Model Trapping: Decisions locked inside session UI.

Repeated Indexing: Every new IDE re-scans codebase.

Token Drain: High API costs re-evaluating files.



Unified Context Layer

Persistent Storage: Central database holds active tasks.

MCP Protocol: Standard tool access across tools.

Instant Handoff: Resume session instantly anywhere.

Model-Agnostic Core Foundations



SurrealDB Engine

Embedded RocksDB persistence with zero external dependencies.



Local Embeddings

Offline `bge-small-en-v1.5` model with HNSW vector indexing.

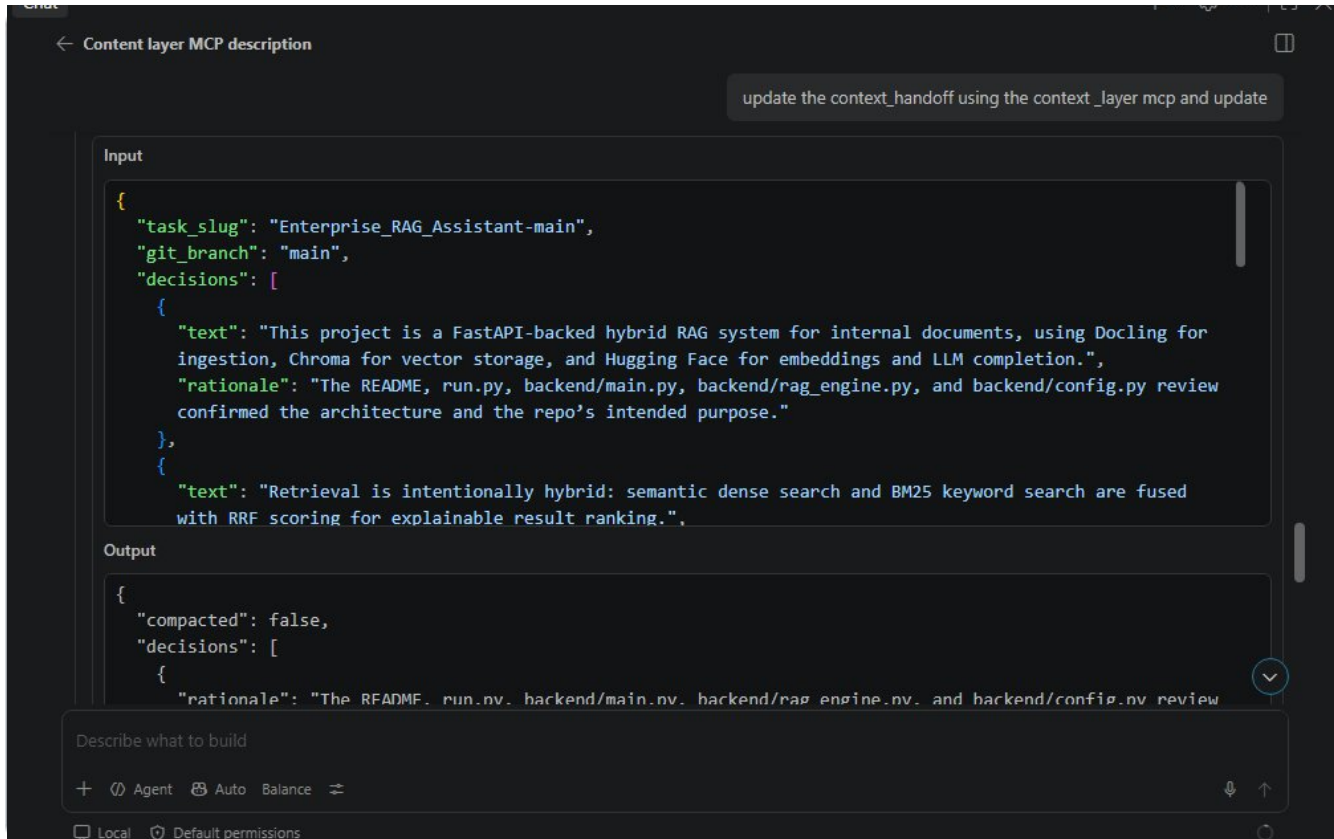


14 MCP Tools

Standardized API for creation, search, and window assembly.

STEP 1: INITIALIZATION

VS Code Copilot: Storing the Handoff



COMMAND EXECUTED

``update context_handoff` via MCP`

- ✓ **Task Slug:** `Enterprise\_RAG\_Assistant-main`
- ✓ **Recorded Decisions:** FastAPI, Docling, ChromaDB, HuggingFace
- ✓ **Vector Generation:** Local embeddings embedded & stored

STEP 2: CROSS-TOOL RESUME

OpenCode: Querying Context Layer MCP

There are existing handoffs for this project. Here's the context:

Task Slug: Enterprise_RAG_Assistant-main

Recent handoff (10:02 UTC):

- Project reviewed: full repo structure mapped
- Architecture confirmed: FastAPI + Chroma + Docling + Hugging Face hybrid RAG
- Key components: upload → Docling markdown → chunk → embed → Chroma + BM25 → RRF → chat with citations
- Config: `.env` for HF key, `settings.json` for models/chunk/weights/RRF

Earlier handoff (08:22 UTC):

- Same file review coverage
- Noted clear separation: `main.py` (routes), `rag_engine.py` (pipeline), `config.py` (settings)

The project is a complete, working hybrid RAG assistant with:

- Document ingestion (PDF, TXT, MD, DOCX via Docling)
- Hybrid search (semantic Chroma + keyword BM25 with RRF fusion)
- RAG chat with source citations
- Live weight tuning UI
- Vanilla JS/CSS/HTML frontend

What specifically would you like to do with this repo? (e.g., run it, fix something, add a feature, audit it)

Ask anything, / for commands, @ for context...

TOOL CALL

``context-layer_search_handoffs``

- ⚡ **Zero Code Re-Scan:** Instant architecture resolution
- ⚡ **Model Used:** Nemotron 3 Ultra Free
- ⚡ **Retrieved Context:** Mapped backend/frontend layout

STEP 3: THIRD-PARTY VERIFICATION

OpenWork: Seamless Handoff Summary

Worked for 1m 47s >

Based on my checks of the MCP (context-layer) handoffs for the `Enterprise_RAG_Assistant-main` project, here's what I found:

MCP Handoff Summary

MCP Server: context-layer (configured in `opencode.json` and `.mcp/config.json`)

Connected Context:

- Task slug: `Enterprise_RAG_Assistant-main`
- Created: 2026-09-22T11:26:57+00:00

Search Results Found:

1. Primary Handoff (relevance 7.92):

- Project overview: Premium hybrid RAG app for internal documentation
- Key files: `README.md`, `run.py`, `backend/main.py`, `backend/rag_engine.py`, `backend/config.py`
- Architecture: FastAPI + Chroma + Docling + Hugging Face
- Current status: Local enterprise RAG assistant focused on internal documentation search with explainable ranking controls

2. Secondary Handoff (relevance 5.77):

- Summary of reviewed files

PROMPT SUBMITTED

"check the handoff from context_layer mcp"

🔄 **Model Used:** Nemotron 3.5 Lightning Free

🔄 **Exact Alignment:** Matches VS Code & OpenCode outputs

🔄 **File Mapping:** Listed ``main.py``, ``rag_engine.py``, ``config.py``

Cross-Platform Fidelity Matrix

Platform	LLM Driver	Action Taken	Context Consistency
VS Code Copilot	Copilot / Local Agent	Created & updated handoff json	Primary Source (100%)
OpenCode	Nemotron 3 Ultra	Fetches via `search_handoffs`	100% Match
OpenWork	Nemotron 3.5 Lightning	Fetches via OpenWork MCP	100% Match

DEMO REPOSITORY ANALYZED

Enterprise RAG Assistant Architecture

Docling Ingestion

Converts PDF, TXT, MD, DOCX directly to structured markdown.

Hybrid Search

Fuses dense Chroma vectors with keyword BM25 scoring.

RRF Ranking

Reciprocal Rank Fusion delivers explainable result scores.

FastAPI Backend

Modular endpoints in `main.py` and `rag_engine.py`.

14 Core MCP Tools Offered

Lifecycle Tools

- ``create_handoff``
- ``resume_handoff``
- ``close_handoff``

Search Tools

- ``search_handoffs``
- ``semantic_search``
- ``get_lineage``

Window Tools

- ``context_for_window``
- ``summarize_window``
- ``budget_report``

Capsule & Skills

- ``get_context_capsule``
- ``list_skills``
- ``get_skill``

Zero-Touch Auto-Compaction



Copilot & Claude

`PreCompact` hooks auto-trigger state saving before context overflows.



OpenCode Plugin

`compaction-plugin.ts` listens for session compaction events.



Generic Loops

`HandoffTrigger` evaluates turn budgets continuously.

Key Advantages of the Context Layer

0

External API Keys Needed

Runs 100% locally with FastEmbed & SurrealDB

Lineage Graph Tracking

Traces task evolution across developer sessions via ``continues_from`` edges.

Multi-Tenant Ready

Namespace isolation via ``CONTEXT_LAYER_TENANT_PREFIX`` for team privacy.



Continuous Context Realized

Eliminate AI agent context fragmentation across VS Code Copilot, OpenCode, and OpenWork today.

```
python -m context_layer.mcp.server
```